

Transformers I: Introduction to Transformers

Natural Language Processing
Master in Applied Artificial Intelligence

Lorena Calvo Bartolomé
lcalvo@pa.uc3m.es

November 23, 2023

Department of Signal and Communications Theory
Universidad Carlos III de Madrid

1. Preliminaries

1.1 Motivation

1.2 The Attention Mechanism

2. The Transformer model

Some tasks we want to solve

Speech
recognition



→ Hey Google,
how it's the weather?

Sentiment
classification

→ The hotel was fantastic!
The hotel was not so good...



Machine
translation

→ How are you?

→ Wie geht's dir?

Machine
summarisation

→ Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Fusce non sem ut dolor
accumsan malesuada.

→ This is a Lorem ipsum.

DNA sequence
analysis



→ TGGAGCTGAGC

Sequence modeling

? What all the former use cases have in common?

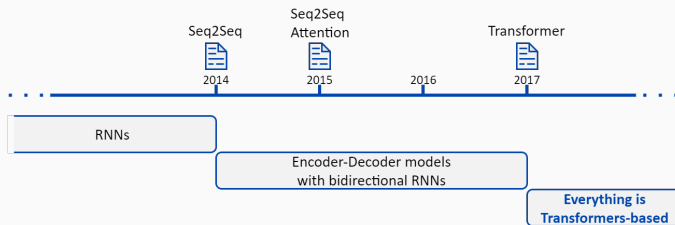
→ The analysis of **sequential data**

Sequence modeling

Field of Deep Learning that deals with the modeling of sequences of data

Examples of sequences

Text sentences, medical signals, speech waveforms, DNA sequences, etc.

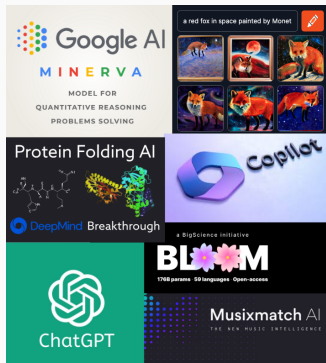
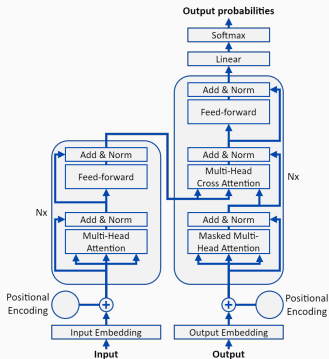


What are Transformers?

Transformers

Deep learning architecture designed for parallel computing on supercomputers, particularly suited for sequence-to-sequence tasks

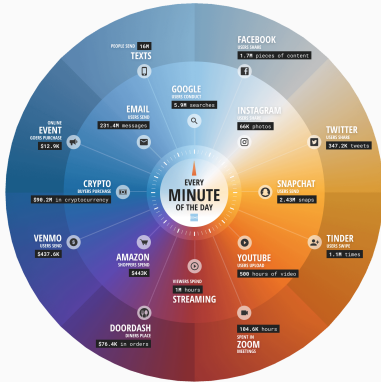
» For years, they have been utilized in many areas, e.g., computer vision, audio, and multi-modal processing, not just NLP



From Transformers to Foundation models

Foundation models

Transformer model that has been trained on supercomputers on billions of records of data and billions of parameters, which can perform a wide range of tasks with no further fine-tuning



- **Examples:** BERT, GPT-3, Llama, etc.
- They were not **created** by academia, but by the **big tech industry**
- **They were born out of necessity:** Big tech had to find a better model to face the exponential increase of petabytes of data flowing into their data centers
- Today they represent the **epitome** of the 4th Industrial Revolution: **The Industry 4.0**

The Impact and Scope of Industry 4.0



Machine to machine connection in manufacturing

Automatic micro-decision making (AI or classical)



Vehicle to warehouse connection in logistics

IoT and all connected devices. Social media server to server connection and processing.



Transportation connections from trucks to ships or airplanes

The industrialization of classical software and of AI with foundation models



- Industry 4.0 requires intelligent algorithms that process data and make decisions without human intervention
- Big techs needed to find an AI model that could perform a variety of tasks that required several separate algorithms in the past

The new paradigm of AI



Foundation models that can do all NLP tasks, CV, and more with one model!

Classical machine learning algorithms (LR, KNN, KMC, MDP, and others)



Partially trained-transformer models that can do one or a few tasks

Expert systems and rule bases when necessary



Classical deep learning tasks when sufficient

Classical coding to help build the AI pipelines



Foundation models constitute a post-deep learning ecosystem with **4 main properties**:

1. Industrial model architecture

2. Huge data sources

3. High computer power

4. Prompt engineering



Transformers brought on a **new sequence modeling paradigm** that:

- by relying only on **attention mechanisms**
- **overperformed** the by then state-of-the-art **RNN-based models**
- while allowing for **parallelization**
- and requiring significantly **less time to train**

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

1. Preliminaries

1.1 Motivation

1.2 The Attention Mechanism

2. The Transformer model

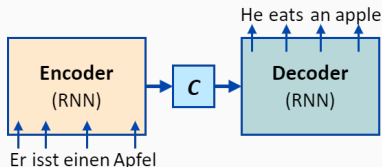
Mechanism for **alignment** in machine translation, image captioning, etc.

- » First presented **in the context of NLP** to **fix the information problem** faced by the early **Seq2Seq** architecture

Seq2Seq

End-to-end model made up of two RNNs:

1. **Encoder**: converts the model's input sequence into a fixed-size representation (context vector)
2. **Decoder**: uses encoder's context vector as “seed” to generate an output sequence



💡 Motivation

A **fixed source representation** (single “context vector”) is suboptimal:

- for the **encoder**, it is hard to compress the sentence
- for the **decoder**, different information from the input may have distinct levels of significance at different time steps

Attention

Mechanism that allows a model to **focus** on different parts of the input at distinct time steps

» *In Seq2Seq*: Single context vector C gets substituted by $\mathbf{C}$, where **each** C_i captures the relevant information for the i -th decoding step

Context vector calculation

1. Calculate decoder' hidden states:

$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$

2. Compute a score for each hidden vector from the original sentence h_j :

$$e_{i,j} = a(s_{i-1}, h_j)$$

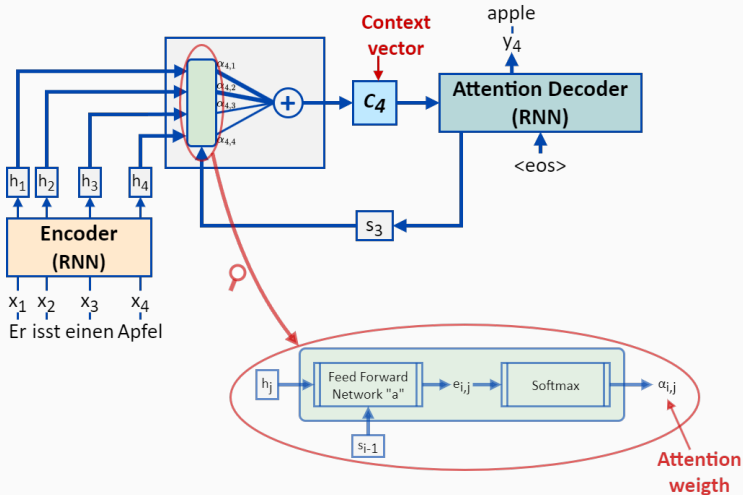
3. Normalize the scores $e_{i,1}, \dots, e_{i,n}$ into a vector $\alpha_i = (\alpha_{i,1}, \dots, \alpha_{i,n})$:

$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^n \exp(e_{i,k})}$$

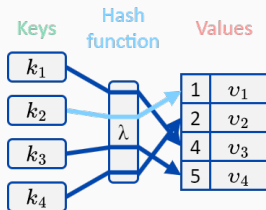
4. Compute c_i as the weighted average of the hidden vectors:

$$c_i = \sum_{j=1}^n \alpha_{i,j} h_j$$

Attention in Seq2Seq



Analogy of the Attention mechanism with retrieval systems

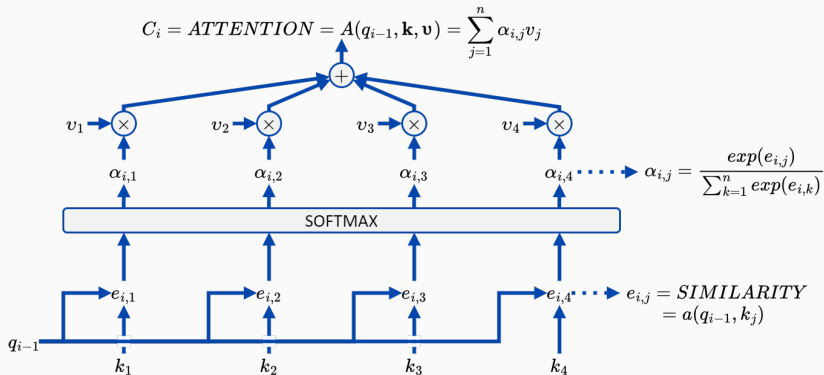


The Attention mechanism mimics the retrieval of **value** v_i for a **query** q based on a **key** k_i in a database lookup table

$$Attention(q, \mathbf{k}, \mathbf{v}) = \sum_i \text{Similarity}(q, k_i) \cdot v_i$$

	Database retrieval	Attention mechanism
Matching	Each query (hash) matches exactly to one query-key pair	Each query matches each key to varying degrees
Similarity(q, K_i)	One-hot vector	Probability vector
Returns	The value corresponding to the found query-key match	Sum of values weighted by the query-key matches

Analogy of the Attention mechanism with retrieval systems



Attention summary

» E.g.: Machine Translation

The query **attends** to the values, i.e. each decoder state (**query**) attends to all the encoder hidden states (**values**)

- **Query:** s_{i-1} , hidden vector for $(i-1)^{th}$ output word
- **Key:** h_j , hidden vector for j^{th} input word
- **Value:** h_j , hidden vector for j^{th} input word

Attention steps

The attention mechanism is two-fold:

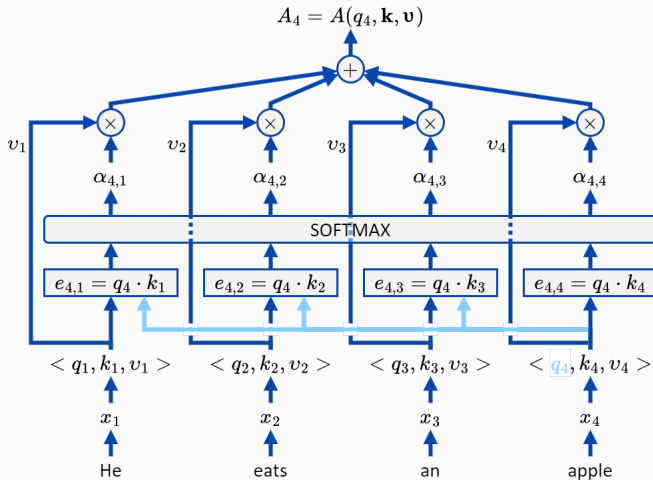
1. Calculate a **similarity score** (weight) measuring the **relevance** between the **query** and each **key-value**
2. Calculate the attention output as the summation of the **values re-weighted** by the similarity scores

Self-Attention

Attention mechanism relating different positions of a single sequence in order to compute a representation of such sequence

	Attention	Self-Attention
<i>Between representations of</i>	Different nature	Same nature
<i>Attention looking FROM</i>	One current decoder state	Each state from a set of states
<i>Attention looking TO</i>	All encoder states	All the states in the same set
<i>Parallelizability</i>	Per-time step words	For all words

Self-Attention example



Self-Attention calculation

1. Calculate **query**, **key** and **value** for each word x_i .
2. Calculate attention score between **query** and **keys**.
3. Normalize attention scores:

$$\alpha_{i,j} = \text{softmax}(e_{i,j}) = \frac{\exp(e_{i,j})}{\sum_{k=1}^n \exp(e_{i,k})}$$

4. Take a weighted sum of **values**:

$$\text{Output}_i = \sum_{j=1}^n \alpha_{i,j} v_j$$

Vectorized Self-Attention calculation

1. Stack all words embeddings in X and calculate **queries**, **keys** and **values**:

$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

2. Calculate attention score between **query** and **keys**:

$$E = QK^T$$

3. Normalize attention scores:

$$A = \text{softmax}(E)$$

4. Take a weighted sum of **values**:

$$\text{Output} = AV$$

1. Preliminaries

2. The Transformer model

2.1 Transformers overview

2.2 Transformer Encoder

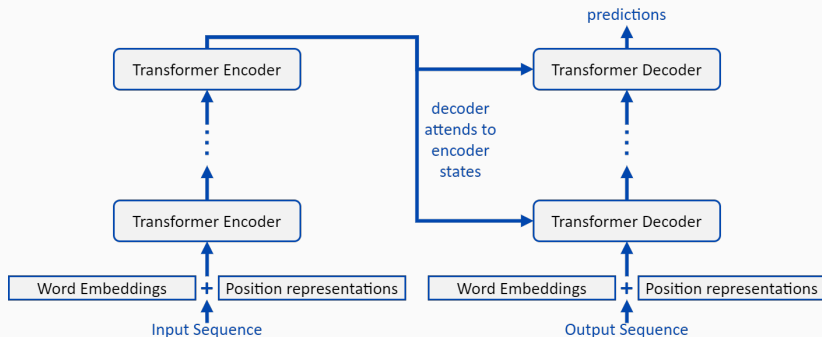
2.3 Transformer Decoder

Transformers (and their variants) are increasingly the model of choice for solving all kinds of NLP tasks

Characteristics

- **Neural network architecture** characterized by an **encoder-decoder** structure that
 - does not imply any RNN
 - **relies only on attention mechanisms**
- Designed to process **sequential data**
 - but the entire input is processed **all at once**, not sequentially
- Allow for **linear parallelization** thus
 - reducing training times
 - facilitating training on larger datasets
- Can handle **long range dependencies**
 - Attention provides context for any position in the input sequence

Transformer model architecture



1. Preliminaries

2. The Transformer model

2.1 Transformers overview

2.2 Transformer Encoder

2.3 Transformer Decoder

Encoder: Self-Attention

ENCODER

WHO is doing:

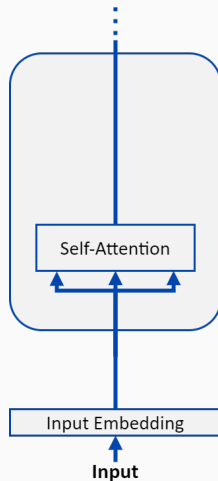
- all source tokens

WHAT are they doing:

- look at each other
 - update representation
- } Repeat N times

💡 Motivation

At each step, allow input tokens to interact with each other (“self”) as a means of understanding reciprocally better in the context of the whole sentence, i.e. to find out where they should put more attention

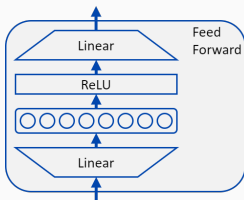


Encoder: Necessities for the Self-Attention (SA) building block

The SA block as it is entails some **problems**:

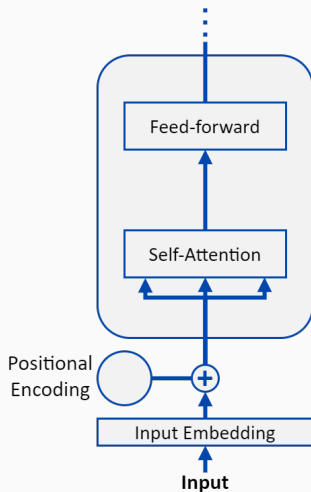
1. **Non-linearities**. SA is performing a re-averaging of the value vectors since there are no element-wise non-linearities

→ **Solution**: Provide non-linear activation by applying a **feed-forward layer** to the attention's output



2. **Sequence order**. SA does not build in order information

→ **Solution**: Encode the order of the sentence in the keys, queries, and values



Positional encoding

The general approach

Represent each **sequence index** as a **vector** $p_k \in \mathbb{R}, k \in 1, 2, \dots, T$ and incorporate such information into the self-attention block:

$$q_i = \tilde{q}_i + p_i \quad k_i = \tilde{k}_i + p_i \quad v_i = \tilde{v}_i + v_i$$

Position vectors calculation

By concatenation of **sinusoidal functions of varying periods**:

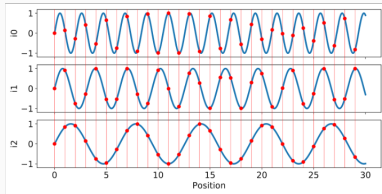
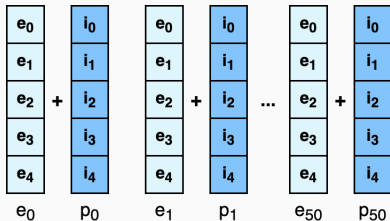
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{n^{\frac{2i}{d}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{n^{\frac{2i}{d}}}\right)$$

where:

- **pos**: Position of a word in the input sequence, $0 \leq k \leq T/2$.
- **d**: Dimension of the output embedding space.
- **n**: User-defined scalar, set to 10000 by the authors.
- **i**: Current dimension in encoding, $0 \leq i \leq d/2$.

Positional encoding



Input
sequence
($d = 4$)

	$i = 0$	$i = 1$	$i = 2$	$i = 3$	
I	$P_{00} = \sin(0)$	$P_{01} = \cos(0)$	$P_{02} = \sin(0)$	$P_{03} = \cos(0)$	$k = 0$
eat	$P_{10} = \sin(1/1)$	$P_{11} = \cos(1/1)$	$P_{12} = \sin(1/10)$	$P_{13} = \cos(1/10)$	$k = 1$
an	$P_{20} = \sin(2/1)$	$P_{21} = \cos(2/1)$	$P_{22} = \sin(2/10)$	$P_{23} = \cos(2/10)$	$k = 2$
apple	$P_{30} = \sin(3/1)$	$P_{31} = \cos(3/1)$	$P_{32} = \sin(3/10)$	$P_{33} = \cos(3/10)$	$k = 3$

Positional Encoding Matrix
($n = 100$)

- Some building blocks are added to help with training
- They do not improve what the model can do, but they are crucial in that they improve the training process.

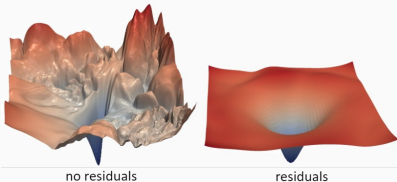
Training tricks

1. **Training trick 1:** Residual connections
2. **Training trick 2:** Layer normalization
3. **Training trick 3:** Scaled Dot-Product Attention

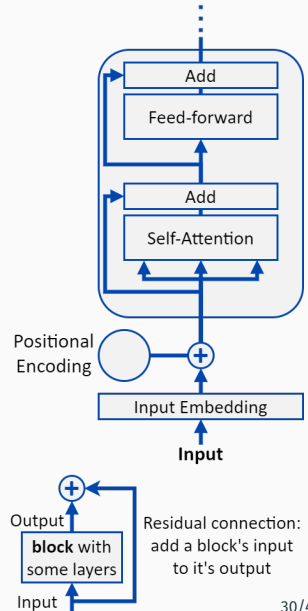
Residual Connections

- **Problem:** Deep networks are bad at learning the identity function
- **Solution:** Pass “raw” embeddings to the next layer so the network does not need to “learn from scratch”:

$$x_l = F(x_{l-1}) + x_{l-1}$$

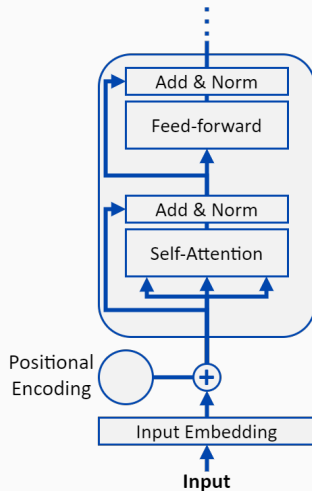
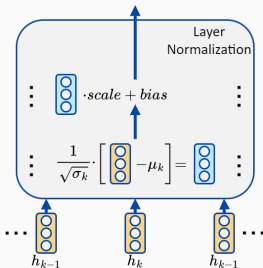


Loss landscape visualization on a ResNet 



Layer Normalization

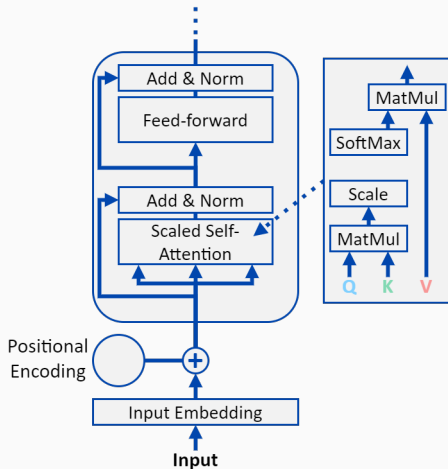
- **Problem:** Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting.
- **Solution:** Reduce uninformative variation by normalizing to zero mean and standard deviation of one within each layer.



Scaled Dot Product Attention

1. **Problem:** After the LN, the mean and variance of vector elements is 0 and 1, respectively, but:
 - The dot product still tends to take extreme values
 - Its variance scales with dimension d_k
 - Input to the softmax can be large, gradients become small
2. **Solution:** Set the variance to 1 by updating the Self-Attention equation:

$$\text{Output} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Encoder: Multi-Head Self-Attention

💡 Motivation

What if we want to look at multiple places in the sentence at once?

» For word i , Self-Attention “looks” where $x_i^T Q^T K x_j$ is high, but maybe we want to focus on different j for distinct reasons?

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions:

- Define **multiple attention “heads”** through various Q, K, V matrices:

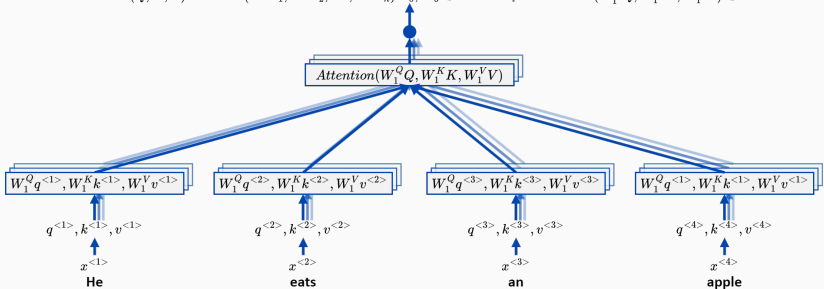
$$\begin{aligned} Q_1 &= XW_1^Q & K_1 &= XW_1^K & V_1 &= XW_1^V \\ &\vdots & & & & \\ Q_l &= XW_l^Q & K_l &= XW_l^K & V_l &= XW_l^V \end{aligned}$$

where $Q_l, K_l, V_l \in \mathbb{R}^{d \times \frac{d}{h}}$, h is the number of attention heads, and l ranges from 1 to h

Encoder: Multi-Head Self-Attention

- Each attention head performs attention independently
- Then the outputs of all the heads are combined:

$$\text{MultiHead}(Q, K, V) = \text{concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) W_0, W_0 \in \mathbb{R}^{d \times d} \text{head}_i = \text{Attention}(W_1^Q Q, W_1^K K, W_1^V V) \in \mathbb{R}^{\frac{d}{h}}$$

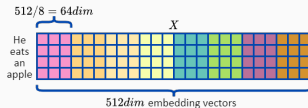


Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality

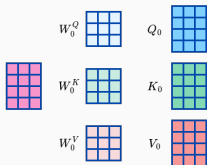
Encoder: Multi-Head Self-Attention



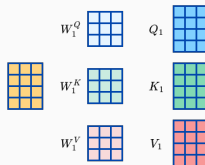
Check Lab “Transformer’s Attention Mechanism”



ATTENTION HEAD #0



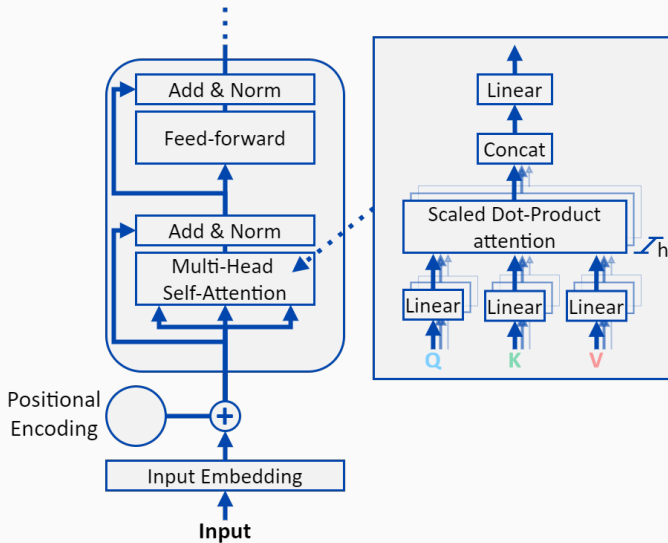
ATTENTION HEAD #1



$$Z_0 = \text{Softmax} \left[\frac{Q_0 K_0^T}{\sqrt{d_k}} \right] V_0 \quad Z_1 = \text{Softmax} \left[\frac{Q_1 K_1^T}{\sqrt{d_k}} \right] V_1$$

$$\begin{bmatrix} Z_0 & Z_1 & \dots & Z_n \end{bmatrix} W_0 = Z$$

Encoder: Complete architecture



1. Preliminaries

2. The Transformer model

2.1 Transformers overview

2.2 Transformer Encoder

2.3 Transformer Decoder

Decoder: Self-Attention

DECODER

WHO is doing:

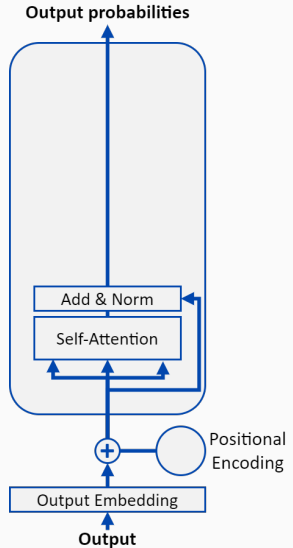
- target token at the current step

WHAT are they doing:

- look at previous target tokens
 - look at source representation
 - update representation
- } Repeat N times

💡 Motivation

At each step, allow tokens of the prefix to interact with each other (“self”), but additionally, look at the encoder states

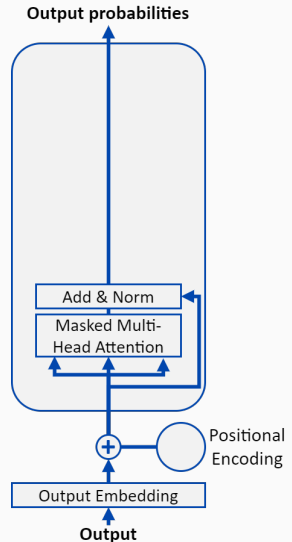


Decoder: Necessities for the Self-Attention Building Block

1. **Masking.** To use SA in the Decoder we need to ensure that we do not look at the future when predicting a sequence
→ **Solution: Masked Multi-Head Attention**, i.e. masking out attention to future words by setting scores to $-\infty$

$$e_{ij} = \begin{cases} q_i^T k_j, & j < i \\ -\infty, & j \geq i \end{cases}$$

	[START]	He	eats	an	apple
[START]	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
He		$-\infty$	$-\infty$	$-\infty$	$-\infty$
eats			$-\infty$	$-\infty$	$-\infty$
an				$-\infty$	$-\infty$
apple					$-\infty$



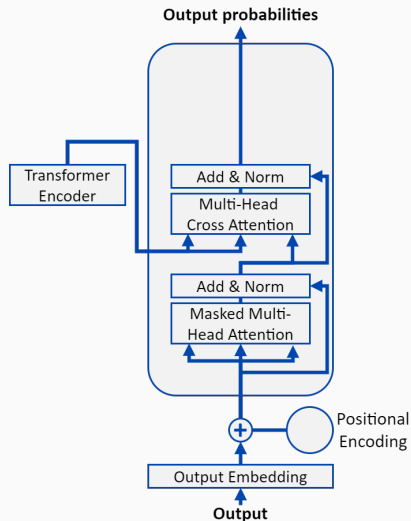
Encoder-Decoder Attention: Multi-Head Cross-Attention

💡 Motivation

Introduce information from the input sequence to the decoder's layers to predict the next output sequence tokens

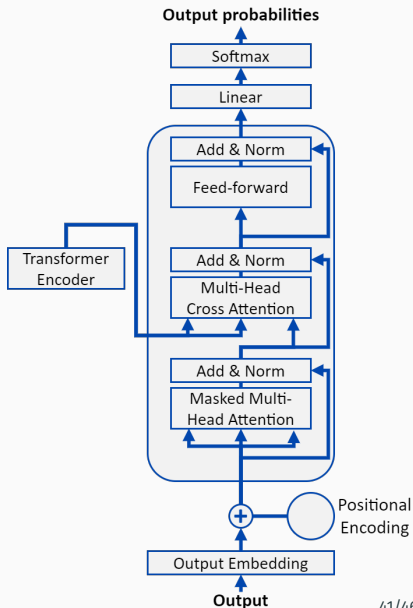
Equivalent to the Encoder's Multi-Head SA, **but for the inputs:**

- Let $h_1, \dots, h_T$ be the output vectors from the Transformer encoder, $x_i \in \mathbb{R}^d$
- Let $z_1, \dots, z_T$ be the input vectors from the Transformer decoder, $z_i \in \mathbb{R}^d$
- **Keys** and **values** are drawn from the **encoder**: $k_i = Kh_i$, $v_i = Vh_i$
- And the **queries** are drawn from the **decoder**: $q_i = QKz_i$

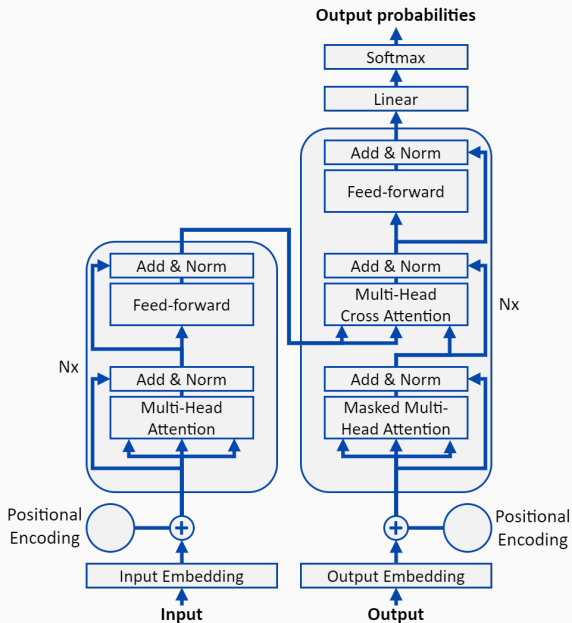


Decoder: Complete architecture

1. Add a feed-forward layer with residual connections and layer norm
2. Add a final linear layer to project the embeddings into a much longer vector of the same length as the vocabulary size
3. Add a final softmax layer to generate a probability distribution of possible next tokens



Transformer Network



Great results with Transformers

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.0	$2.3 \cdot 10^{19}$	

Results on Machine Translation (📄): Better scores and more efficient training!

Model	Test perplexity	ROUGE-L
<i>seq2seq-attention</i> , $L = 500$	5.04952	12.7
<i>Transformer-ED</i> , $L = 500$	2.46645	34.2
<i>Transformer-D</i> , $L = 4000$	2.22216	33.6
<i>Transformer-DMCA</i> , no MoE-layer, $L = 11000$	2.05159	36.2
<i>Transformer-DMCA</i> , MoE-128, $L = 11000$	1.92871	37.9
<i>Transformer-DMCA</i> , MoE-256, $L = 7500$	1.90325	38.8

Results on document generation (📄): Transformers dominate across the board!

- Transformers' parallelizability allows for efficient pretraining, and have made them the de-facto standard.



Bond, F., Isahara, H., Fujita, S., Uchimoto, K., Kuribayashi, T., & Kanzaki, K. (2009). Enhancing the Japanese Wordnet. *Proceedings of the 7th Workshop on Asian Language Resources*, 1–8.



Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.



Godoy, D. V. (2022). *Deep learning with pytorch step-by-step, volume iii: Sequences & nlp*. Independently published.



Joshi, M., Chen, D., Liu, Y., Weld, D. S., Zettlemoyer, L., & Levy, O. (2020). Spanbert: Improving pre-training by representing and predicting spans. *Transactions of the Association for Computational Linguistics*, 8, 64–77.



Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

Bibliography ii



Manning, C., Socher, R., Fang, G. G., & Mundra, R. (2017). Cs224n: Natural language processing with deep learning¹.



Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. (2018). Improving language understanding by generative pre-training.



Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(8), 9.



Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P. J., et al. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer.. *J. Mach. Learn. Res.*, 21(140), 1–67.



Rothman, D. (2022). *Transformers for natural language processing - second edition*. Packt Publishing Ltd.



Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

 Voita, E. (2020). NLP Course For You.
https://lena-voita.github.io/nlp_course.html

 Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2021). Dive into deep learning. *arXiv preprint arXiv:2106.11342*.